Ders 3 Genişletilmiş Notu: Gözden Geçirme, Fagan İncelemesi, V Model ve Birim Testi

Ders 3 Genişletilmiş Notu: Gözden Geçirme, Fagan İncelemesi, V Model ve Birim Testi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Gözden Geçirmenin Tekrarı
- 2. Gözden Geçirme Tipleri ve Seçenek Matrisi
- 3. Süreç Basamaklarına Göre Gözden Geçirme
- 4. Gözden Geçirme Standartları
- 5. İnceleme (Inspection) ve Statik Doğrulama
- 6. Genel İnceleme Süreci
- 7. Fagan İncelemesi
- 8. İncelemenin Tekrarı ve Ara Soru-Cevap
- 9. Test Nedir?
- 10. Doğrulama ve Geçerleme Terimleri
- 11. V Model Yaklaşımı
- 12. Test Süreci Adımları ve Birim Testi
- 13. Bütünlük Testi ve Regresyon Testi
- 14. Kabul, Alfa, Beta ve Teşhis Testi
- 15. Birim Testi Üzerine Soru-Cevap

1. Gözden Geçirmenin Tekrarı

Hoca derse, kaliteyi sağlamak için gereken aktivitelerden gözden geçirmeyi (review) tekrar edeceğini ve ardından inceleme (inspection) üzerinde duracağını söyleyerek başladı. Bundan sonraki derslerde yalnız bir önceki dersi tekrar edeceğini, dönemin başından beri işlenenlerin hepsini yeniden anlatmayacağını belirtti.

Gözden geçirme ve onaylamada her basamak tamamlanınca durak noktalarında (milestone) yapılan işler gözden geçirilir ve gerekli düzeltmeler yapılır.

1.1 Hataların Kaynağı ve Maliyet

Bağlantı geri geldiğinde hoca hataların nerede yapıldığına dair bir sayı verdi. Yazılım geliştirmedeki hataların yaklaşık %65'i tasarım aşamasında yapılır. Gözden geçirme sırasında bu hataların yaklaşık %75'i bulunup düzeltilebilir. Gözden geçirmede bulunamayan hatalar da test (sınama) öncesinde bulunursa daha az maliyete yol açar. Sürecin ilerisine gidildikçe hatayı düzeltmenin maliyet katsayısı artar.

> **Ders sonrası not:** Hoca bu oranı önce "%50'si, 65'i" diye söyleyip düzeltti. Kısa nota göre slayttaki değer %65'tir; altyazıdan slaytın kendisi doğrulanamıyor, değer büyük olasılıkla %65'tir.

1.2 Gözden Geçirme Nedir?

Gözden geçirme, yazılımda yapılabilen işlevsel, mantıksal ve gerçekleştirme (kodlama) hatalarının bulunup giderilmesidir. Hoca, hataların bulunması aşamasının asıl gözden geçirme olduğunu vurguladı. Gözden geçirmenin ek faydaları şunlardır:

- Yazılımın gereksinimlere uygun düzenlendiğinin onaylanması.
- Yazılımın önceden belirlenen standartlara uygunluğunun sağlanması.
- Yazılımın düzenli olarak geliştirilmesi.
- Yazılım proje yönetiminin kolaylaşması.
- Müşteriye dağıtılacak ürün belgelerinin kalite ve miktar bakımından iyileşmesi.

Bu nedenle gözden geçirme her aşamada yapılmalıdır. Hoca, yazılım geliştirme süreç modellerinde gözden geçirmenin her zaman "bir adım bitince" yapılmadığını, modele göre sürecin farklı yerlerine oturtulduğunu hatırlattı.

Gözden geçirme, teknik uzmanların çalışmalarının yine teknik uzmanlarca değerlendirildiği bir toplantıdır. Yazılım mühendisliği sürecinde üretilen ürünlerin teknik değerlendirmesidir. Hocaya göre gözden geçirme ve onun bir kademe ileri hali olan inceleme, yazılım kalite sağlama aktivitelerinin en önemlileridir. Gözden geçirme aynı zamanda bir eğitim ortamıdır.

Geçen hafta görülen "gözden geçirme ne değildir" listesi de tekrarlandı:

- Bir proje özeti veya proje ilerleme sunumu değildir.
- Yazılım geliştiricilerin yalnızca bilgi verdiği bir toplantı değildir.
- Yazılım ekibinin politik ve kişisel değerlendirmesi değildir.

1.3 Gözden Geçirme Toplantısındaki Roller

- **Gözden geçirme lideri:** Toplantıyı yönetir. Ayrı bir lider yoksa katılımcılardan biri toplantıyı idare edebilir.
- **Kullanıcı temsilcisi:** Müşteri tarafından, kullanım gereksinimlerini takip eden kişidir. Üretilen ürünün, sistem analistleriyle birlikte belirlenen gereksinimlere uygun olup olmadığını kontrol eder.
- **Kalite güvence ekibinden biri:** Ürünün kurum standartlarına uygun hazırlanıp hazırlanmadığını kontrol eder. Kurumda kalite güvence ekibi yoksa standartları belirleyen yazılım ekibi bu açıdan gözden geçirmeyi üstlenir.
- **Kayıtçı:** Katılımcılardan biri kayıt tutar.
- **Üretici:** Yazılım geliştirme ekibinden biridir. Yapılan iş hakkında tanımlama verir. Hata bulunmuşsa hatanın nasıl giderileceğine dair not alır.

2. Gözden Geçirme Tipleri ve Seçenek Matrisi

Gözden geçirme tipleri, basit ve kuralsız (formal olmayan) gözden geçirmeden başlayıp daha kurallı, düzenli bir ekiple yapılan gözden geçirmeye ve en ayrıntılı tip olan incelemeye kadar uzanır. Tipler şunlardır:

- Formal olmayan gözden geçirme (informal review). Derste "review" adı bunun için kullanılıyor.
- Yapısal denetim (walkthrough).
- İnceleme (inspection).
- Round robin peer review.

Hoca geçen hafta görülen tabloyu yeniden gözden geçirdi:

Özellik	Formal olmayan GG	Walkthrough	Inspection
Gözden geçirme lideri	yok	var	var
Toplantı takvimi	olabilir, basit bir takvim	var	var
Gözden geçirenlerin ön hazırlığı	belki	var	mutlaka var

Özellik	Formal olmayan GG	Walkthrough	Inspection
Yazılımcının sunum yapması	belki, kesin değil	var	yok
Hataları bulmak için kontrol listesi (checklist)	yok	yok	mutlaka var
Hataların sınıflanıp sayılması	yok	yok	mutlaka var
Kayıtçı	belki	var	var

Tablodaki bazı satırlar derste şöyle açıklandı:

- **Ön hazırlık:** Kodların ya da raporların toplantıdan önce paylaşılıp incelenmesidir.
- **Yazılımcının sunumu:** Walkthrough'da yazılımcı önden bir sunum yapar. Inspection'da sunum yoktur, doğrudan ürünler üzerinde inceleme yapılır.
- **Kontrol listesi:** "Hatalar hangi gruptadır, şu grupta mıdır?" diye işaretlenen bir listedir. Inspection'da mutlaka hazırlanır.
- **Hata sınıflaması:** Bulunan hataların "şu tip, bu tip" diye sınıflanması ve her sınıfta kaç hata olduğunun belirtilmesidir. Yalnız inspection'da yapılır.

Gözden geçirmeyi kimin yaptığı projenin büyüklüğüne göre değişir. Küçük projelerde gözden geçirme ve düzeltme doğrudan yazılım geliştirici tarafından yapılır. Orta ve büyük projelerde hata ve eksiklikleri genellikle üçüncü bir kişi bulur.

3. Süreç Basamaklarına Göre Gözden Geçirme

Gözden geçirme ve onaylama, yazılım süreç basamaklarına göre yapılır. Basamaklar şunlardır: sistem analizi, yazılım geliştirme planı, gereksinim analizi, tasarım, kodlama, test (sınama), bakım ve onarım. Bu basamakların bittiği noktalar durak noktaları ya da kilometre taşlarıdır (milestone). Hoca her basamakta gözden geçirmede neye bakıldığını sırayla anlattı.

3.1 Sistem Analizi

Sistem düzeyindeki spesifikasyonların yazılıma ilişkin işlev ve görevler olarak dağıtımının doğruluğu, kalite güvencesini onaylama gereksinimlerinin tutarlılığı ve görevlendirmenin yeterliliği denetlenir. Sistem analizi sırasında ana işlevler, ana ara birimler (interface), performanslar ve tasarım kısıtları tanımlanır. Gözden geçirmede şu sorulara değinilir:

- Farklı seçimlerle tutarlılık nasıl değişir?
- Sistem teknolojik bakımdan ne kadar yapılabilir (feasible)?
- Sistemin testinde hangi esaslara önem verilecek?
- Sistem öğeleri arasında nasıl bir uyum var?

3.2 Yazılım Geliştirme Planı

Bu basamakta özellikle projenin gerçekleştirilmesindeki risk üzerinde durulur ve sorun tanımlanır. Planda belirtilen şu konular gözden geçirmede incelenir ve değerlendirilir:

- Terimler doğru mu?
- Kaynaklar yeterli mi, sağlanabilecek mi?
- Görevler nasıl paylaştırılacak? Görevlerin uygunluğu ve sırası nedir?
- Verimlilik, kalite, gider ve zamanlama tahminleri. Proje belirtilen zamanda bitirilebilecek mi?
- Seçilen kişiler bu işe uygun mu?

3.3 Gereksinim Analizi

Gereksinimlerin birbiriyle tutarlı olup olmadığına ve gerçek hayattaki isteklere bağlı olup olmadığına bakılır. Gereksinimlerin zaman, kaynak ve bütçe olanakları bakımından gerçekleştirilebilir olması dikkate alınır. Onaylama kriterlerinin doğru konup konmadığı da incelenir.

3.4 Tasarım

Ön tasarımda gereksinimlerin tasarıma doğru dönüştürülüp dönüştürülmediğine bakılır. Burada yazılım mimarisi, yani yazılımın bütününün hangi alt parçalarla oluşturulacağı ve bu bütünün doğru tasarıma gidip gitmediği incelenir. Ön tasarımda önemli olanlar şunlardır:

- Gereksinimlerin yazılım mimarisine doğru yansıtılması.
- Modüllerin etkinliği.
- Modüller arasındaki işlevsel bağımsızlık.
- Program yapısının uygunluğu.

Ayrıntılı tasarımda ise veri yapılarının ve işlemlerin doğru olup olmadığı kontrol edilir.

3.5 Kodlama (Gerçekleme)

Kodlamada ayrıntılı tasarım programlama diline dönüştürülür ve bu dönüşüm sırasında da hata yapılabilir. Kodlamanın izlenmesinde şunlara bakılır:

- Tasarımın kaynak programa dönüştürülmesinin doğruluğu.
- Yazılım hataları.
- Kodlama standartlarına, yani kullanılan programlama dilinin standartlarına uygunluk.
- Açıklamaların (yorum satırlarının) doğru ve tutarlı olması.
- Tasarım hatalarının düzeltilip düzeltilmediği.

Hoca, kodlama sırasındaki gözden geçirmenin daha çok review yerine kod incelemesi (code inspection) ile yapıldığını belirtti.

Bu bölümde ekran paylaşımı tasarım slaytında takılı kaldı. Hoca slaytı yeniden paylaştı ve böyle durumlarda öğrencilerin hemen haber vermesini istedi.

4. Gözden Geçirme Standartları

Gözden geçirmenin nasıl yapılacağını tanımlayan standartlara örnekler verildi:

- Sistem tasarımı gözden geçirmesi (system design review).
- Yazılım spesifikasyonu gözden geçirmesi (software specification review).
- Ön tasarım gözden geçirmesi (preliminary design review).
- Teste hazır olmanın değerlendirilmesi (test readiness review): testten önce yazılımın test edilmeye hazır olup olmadığının değerlendirilmesi.

5. İnceleme (Inspection) ve Statik Doğrulama

Hoca bugünkü konuyu soru başlıklarıyla özetledi: Yazılım kalite değerlendirmede incelemenin yeri nedir? Test nedir? İnceleme nedir? İnceleme ekibinin yapısı ve rolleri nelerdir? Teknik inceleme raporunda neler bulunur?

Yazılım incelemesi, testle birlikte yazılım kalitesini sağlamak için en çok kullanılan yöntemlerden biridir. Testin aksine hataları **kodu çalıştırmadan** bulma esasına dayanır. Bu yüzden statik doğrulama (static verification) yöntemi olarak tanımlanır.

5.1 Neler İncelenebilir?

Yazılımın bütün ürün ve yarı ürünleri incelenebilir:

- Gereksinim spesifikasyonu.
- Yüksek seviye (genel) tasarım: modül haritası, modül sınıflaması doğru oluşturulmuş mu?
- Formal spesifikasyon.
- Ayrıntılı tasarım.
- Program kodu.

Test ise yalnız çalıştırılabilen şeyler üzerinde yapılır: program ya da prototip. Gereksinim spesifikasyonu veya tasarım sırasında bir prototip hazırlanmışsa bu prototip çalıştırılarak test edilebilir. Gereksinim spesifikasyonu incelenirken bağlantılı prototip de incelenebilir. Ama yazılı

olarak hazırlanmış bir gereksinim spesifikasyonu ya da ayrıntılı tasarım belgesi çalıştırılarak test edilemez. Bunlar üzerinde inceleme yapmak gerekir.

```
Statik doğrulama (inceleme, inspection):
  Gereksinim spesifikasyonu, yüksek seviye tasarım, formal spesifikasyon,
  ayrıntılı tasarım, program

Dinamik doğrulama ve geçerleme (test):
  Prototip, program
```

İnceleme bir doğrulamadır (verification), çünkü gidiş yolunun doğruluğuna bakar: Gereksinim spesifikasyonunu doğru oluşturduk mu? Tasarımı doğru hazırladık mı? Formal spesifikasyonu doğru yaptık mı? Test ise ürün çalıştırılarak yapıldığı için dinamik doğrulama ve geçerlemedir (dynamic verification and validation).

6. Genel İnceleme Süreci

Basit inceleme süreci üç ana aşamaya ayrılır:

1. **Planlama ve hazırlık.** İncelemeyle ilgili genel sorular cevaplanır: İncelemenin amacı ve hedefleri nelerdir? Hangi yazılım parçaları incelenecek? İncelemeyi kim yapacak ve bu kişiler hangi özellikleri taşımalı? Süreç boyunca hangi inceleme teknikleri ve takip eylemleri uygulanacak? İnceleme çalışmaları planlanır ve gerekli altyapı varsa sağlanır.
2. **İnceleme ve toplantı.** Herkes kendine düşen kodu ya da gereksinim raporundaki bölümü belirlenen kriterlere göre inceler. Bulunan hatalar bir sonraki aşamada düzeltilmek üzere kayda alınır. Sonra ekip bir araya gelir ve hataların nasıl düzeltileceğine karar verilir. Hoca bu aşamada inceleme ile toplantının bir arada olduğunu, çünkü toplantı sırasında da inceleme yapıldığını söyledi.
3. **Düzeltme ve tamamlama.** Bulunan hatalar sorumlu kişilerce düzeltilir ve takip edilir. Düzeltmeyi kimin yapacağı belirlenir, iş tamamlanır ve raporlanır. İstenen sonuç elde edilemezse sürecin başına dönülür ve yeniden inceleme yapılır.

7. Fagan İncelemesi

Hoca, Fagan yönteminin çok eski olduğunu, ama bugün de “inspection” denince bilinen ve ayrıntılı inceleme için süreç tanımı olarak kullanılan bir yöntem olduğunu söyledi. Yöntem özellikle nasıl bir ekip kurulacağını ve hangi adımlarda hangi işlerin yapılacağını tanımlar. İnceleme için farklı kişilerin önerdiği çalışma biçimleri vardır; bunların en çok kullanılanı Fagan’ınkidir.

7.1 Fagan İncelemesinin Adımları

Fagan yöntemi, genel inceleme sürecini daha fazla alt adıma böler:

1. **Planlama.** İncelemenin ne zaman tamamlanacağı ve ekibin ne zaman toplanacağı planlanır.
2. **Planın gözden geçirilmesi.** Verilen görevler ve herkese paylaştırılan kod inceleme bölümleri gözden geçirilir. Planın doğru olup olmadığı kontrol edilir.
3. **Kişisel kod inceleme.** Herkes kendine verilen bölümü inceler.
4. **İnceleme toplantısı.** Kişisel incelemeler bitince ekip toplanır.
5. **Düzeltme.** Toplantıda ortaya çıkanlar düzeltilmesi için yazılım ekibine gönderilir.
6. **Tamamlama ve raporlama.**

   Ek bilgi (derste anlatılmadı): Fagan yöntemi büyük olasılıkla 1970’lerin ortasında IBM’de Michael Fagan tarafından tanımlanmıştır. Kaynaklarda adımlar büyük olasılıkla İngilizce olarak planning, overview, preparation, inspection meeting, rework ve follow-up diye geçer. Özgün tanımda ikinci adım (overview) büyük olasılıkla incelenecek ürünün ekibe kısaca tanıtıldığı bir toplantıdır; hoca bu adımı “planın ve görev paylaşımının gözden geçirilmesi” olarak anlattı.

Fagan incelemesini diğer inceleme süreçlerinden ayıran özellikler şunlardır: raporlama ve toplantı yapılış biçiminin kurallı olması, ön hazırlık yapılması ve kimin neyi inceleyeceğine

ekip yapısı içinde karar verilmesi.

7.2 Etkili Kod Okuma

Fagan incelemesinde kod okuma önemlidir. Etkili bir kod okuma tekniği, yukarıdan aşağıya (top-down) ve aşağıdan yukarıya (bottom-up) incelemenin birleşimi olmalıdır. Yani hem en genel program modülünden başlayıp onun çağırdığı modüllere doğru inilir, hem de alttaki ayrıntı modülleri ayrıntılı olarak incelenir.

Program mümkün olduğu kadar küçük parçalara (programlara, prosedürlere, alt programlara) ayrılır. Önce her parçanın kendi içindeki bütünlüğüne bakılır. Büyük resim kaybedilmeden, programın genel yapısı hep göz önünde tutularak en küçük parçaların doğruluğu denetlenir.

7.3 İnceleme Ekibi

İnceleme ekibi şu kişilerden oluşur:

- İnceleme sorumlusu.
- İnceleyiciler: kod okuma ve inceleme işini yapanlar.
- Kodu yazan.
- Kayıtçı: rapor yazan, kayıt tutan kişi.

7.4 Teknik İnceleme Özet Raporu

İnceleme sonunda bir teknik inceleme özet raporu hazırlanır. Raporda mutlaka şunlar açıklanmalıdır:

- Neyin gözden geçirildiği.
- Kimin gözden geçirdiği: her alt modülün, alt bölümün kime verildiği eşleştirilerek yazılır.
- Hangi hata ve eksiklerin bulunduğu ve ne yapılması gerektiği.

Rapor şu üç karardan biriyle sonuçlanır:

- Kod değişiklik yapılmadan kabul edilir.
- Belirtilen önemsiz hataların düzeltilmesi koşuluyla kabul edilir.
- Çok sayıda önemli hata nedeniyle reddedilir; düzeltme istenir ve düzeltme sonunda yeniden inceleme yapılır.

Hoca ardından geçen hafta görülen standartları da hatırlattı: Yazılım geliştirmeye başlarken yazılımın hangi standartlara uyacağı, kurumun hangi standartları kabul ettiği kalite el kitabında tanımlanır ve böylece kurumun standart yapısı belirlenir.

8. İncelemenin Tekrarı ve Ara Soru-Cevap

Öğrenciler slaytların bir süredir görünmediğini bildirince hoca inceleme konusunu slaytlar eşliğinde bir kez daha özetledi. Bu tekrar, yukarıdaki 5-7. bölümlerdeki bilgileri yineledi; yeni olarak şu vurgu geldi: İnceleme bir “verification”dır, çünkü yazılımı geliştirirken gidiş yolunun doğru olup olmadığına, ürünü doğru yolla oluşturup oluşturmadığımıza bakar. Statik doğrulama bütün yarı ürünler üzerinde yapılabilir. Test ise ürün çalıştırılarak yapılan dinamik doğrulama ve geçerlemedir.

Test konusuna geçmeden önce hoca, gözden geçirme ve incelemenin anlaşılıp anlaşılmadığını görmek için kısa bir soru-cevap yaptı.

Soru: İnceleme ile gözden geçirme arasındaki farklar nelerdir? **Cevap (öğrenci):** İnceleme gözden geçirmeye göre daha ayrıntılıdır, konunun daha derinine girilir. Bir öğrenci incelemede çözüm üretmek için yollar arandığını, gözden geçirmede ise yalnız fikirlerin oluşturulduğunu söyledi.

Hoca ardından seçenek matrisini hatırlatarak “önceden hazırlık yapılır mı, rapor hazırlanır mı” gibi farklar sordu.

Cevap (öğrenci): Gözden geçirmede sunum hazırlanmıyor, incelemede hazırlanıyor ve konu daha ayrıntılı inceleniyor. Kod üzerinden düşünürsek gözden geçirmede hatalar kodu test etmeden, incelemede ise test ederek bulunuyor.

Hoca bu cevaplara ayrıntılı bir düzeltme yapmadı; yalnız gözden geçirmenin yazılım geliştirmenin her aşamasında yapılabildiğini ekledi.

> **Ders sonrası not:** Öğrenci cevaplarındaki üç ifade derste anlatılanlarla çelişir. (1) Seçenek matrisine göre yazılımcının sunum yaptığı tip walkthrough'dur; inspection'da yazılımcı sunum yapmaz, doğrudan ürünler incelenir. (2) İnceleme kodu çalıştırmadan yapılan statik doğrulamadır; kodu çalıştırarak hata bulmak testtir. (3) İncelemenin amacı hataları bulmak, sınıflamak ve kaydetmektir; düzeltme ayrı bir adımda geliştirici ekibe bırakılır.

Soru: Statik doğrulama (static verification) nedir? Gözden geçirme, inceleme ve testten hangisi statik doğrulamadır? **Cevap:** Programı çalıştırmadan yapılan değerlendirmedir. İnceleme statik doğrulamadır.

Soru: Dinamik doğrulama (dynamic verification) hangisidir? **Cevap:** Bir öğrenci "test", bir başkası onun özel bir hali olan "birim testi (unit test)" dedi. Hoca birim testinin doğru ama eksik olduğunu söyledi: Birim testinde kod çalıştırılıp istenen işi yapıp yapmadığına bakılır. Ama bütünlük testinde ve sistem testinde de program çalıştırılır. Bu yüzden program parçasını ya da bütününü çalıştırarak yapılan bütün test işlemleri dinamik doğrulamadır.

9. Test Nedir?

Test, bir programdaki hataları bulmak amacıyla yapılan işlemlerdir. Test, yazılımın dört açıdan yeterliğini denetler:

- **İşlevsel (functional):** Gereksinim analizinde tanımlanan bütün işlevler yerine getiriliyor mu? Tek bir modüle, yani birim testi çerçevesinde bakıldığında da modül kendisine biçilen en küçük görevi doğru yapıyor mu?
- **Performans:** İstenen işi istenen performansta ve verimlilikte yapıyor mu?
- **Dayanıklılık:** Belli özellikler tanımlanmışsa yazılım bu özelliklerin sınırlarında da dayanıklı mı?
- **Yapısal:** İç algoritma tanımı açısından yeterli mi?

Soru (bir öğrencinin, büyük olasılıkla sohbet penceresinden): Performans ve dayanıklılık işlevsel olmayan gereksinimlere mi girer? **Cevap:** Evet, performans ve dayanıklılık işlevsel olmayan (non-functional) gereksinimlerdir. V model yaklaşımında işlevsel olmayan gereksinimler sistem testinde test edilir. Gereksinim analizinde belirtilen işlevsel olmayan gereksinimlerin sağlanıp sağlanmadığı sistem testinde anlaşılır.

10. Doğrulama ve Geçerleme Terimleri

Hoca V model slaytında terimleri değiştirdiğini anlattı. Test alanında çalışanların yaptığı son toplantıda "verification" için **doğrulama**, "validation" için **geçerleme** karşılıkları kabul edilmiştir.

- **Doğrulama (verification):** "Ürünü doğru yolla üretiyor muyuz?" Ürünü elde etmek için geçilen süreçte yapılan işlemlerin denetlenmesidir.
- **Geçerleme (validation):** "Elde ettiğimiz yazılım ürünü doğru ürün mü?"

Hoca, sözlükte her iki İngilizce sözcüğe de hem "doğrulama" hem "geçerleme" karşılığının verildiğini söyledi. Kavram sözlüğü henüz Türkçeleştirilmemişken tam tersini kullandıklarını, ama genel kabul bu yönde olduğu için slaytları düzelttiğini belirtti.

> **Ek bilgi (derste anlatılmadı):** Bu ayrım İngilizce kaynaklarda büyük olasılıkla Barry Boehm'e atfedilen iki soruyla özetlenir: verification "Are we building the product right?", validation "Are we building the right product?". Hocanın Türkçe tanımları bu iki soruyla örtüşür.

11. V Model Yaklaşımı

V model, yazılım geliştirme sürecindeki kontrolleri ürün üzerinden değerlendirir ve hangi adımda doğrulama, hangi adımda geçerleme yapıldığını gösterir. Model adını hem şeklinden hem de verification ve validation sözcüklerinin baş harfi olan V'den alır.

11.1 Modelin İki Kolu

V'nin **sol kolu** yazılım geliştirme aktiviteleridir. Sol üstten başlanır ve aşağı inilir:

- **Müşteri gereksinimleri:** Müşterinin hangi problem nedeniyle yazılımı istediği ve yazılımdan beklentileri madde madde anlatılır. Şekilde elips içinde, müşterinin anlattığı istekler olarak gösterilir.
- **Ürün spesifikasyonu:** Ürünün işlevsel ve işlevsel olmayan gereksinimlerle, yazılım geliştiricilerle birlikte yapılmış formal tanımıdır. Şekilde dikdörtgen içinde, sistem analistinin ya da geliştiricinin anlayıp düzenlediği gereksinimler olarak gösterilir. Yazılım gereksinim analizi sırasında oluşturulur.
- **Yüksek düzey tasarım:** Genel ya da mimari tasarımdır. Modüllerin hiyerarşisi ve yapı diyagramları (structure diagram) burada oluşturulur.
- **Alçak düzey tasarım:** Her modülün iç yapısı, algoritması, arayüzleri ve veri tabanı tasarımıdır.
- **Kodlama:** Alçak düzey tasarımda belirtilen birimlerin kodlanmasıdır. Kodlama ve birim testi V'nin dibindedir.

V'nin **sağ kolu** test süreci adımlarıdır ve aşağıdan yukarı ilerler. Her küçük kod birim testiyle test edilir. Birleştirme kademeli yapılırsa sırayla bileşen testi, bütünlük testi ve sistem testi gelir. En üstte müşteri kullanım testi, yani işlemsel kullanım aşaması vardır.

11.2 Adımların Eşleşmesi

V modelin amacı, yazılım geliştirme süreç adımlarıyla test süreci adımlarını eşleştirmektir. Hocanın anlattığı eşleşme şöyledir:

```
Müşteri gereksinimleri  ---- Geçerleme -------------->  İşlemsel kullanım
  Ürün spesifikasyonu   ---- Geçerleme ve doğrulama -->  Sistem testi
    Yüksek düzey tasarım ---- Doğrulama ------------>  Bütünlük testi
      Alçak düzey tasarım ---- Doğrulama --------->  Bileşen testi
                Kodlama ---- Doğrulama ---->  Birim testi
```

- **Alçak düzey tasarım:** Ayrıntılı tasarım hem birim testinde hem bileşen testinde test edilir. Alçak düzey tasarımda algoritmalar da test edilmiş olur. Burada doğrulama yapılır.
- **Yüksek düzey tasarım:** Bütünlük testiyle karşı karşıya gelir. Bütünlük testi yapılırken, genel tasarımda oluşturulan modül hiyerarşisi ve yapı diyagramlarıyla gösterilen modül yapısı kontrol edilir: Gerekli parçalar doğru birleştirilebilmiş mi? Burada da doğrulama yapılır.
- **Ürün spesifikasyonu:** Sistem testinde gereksinim analizi ve spesifikasyonu kullanılır. Hem "bu işi doğru yürüttük mü?" hem "doğru ürünü elde ettik mi?" sorulur. Bu yüzden sistem testinde hem doğrulama hem geçerleme yapılır. Sistem testi ayrıca sistemin bütün olarak doğru çalışmasının ötesinde, uygun bir sistem ortamında nasıl çalışacağı sorusuna da cevap arar.
- **Müşteri gereksinimleri:** İşlemsel kullanımda müşterinin istekleri, iş analistinin aktardığı haliyle kullanılır: Müşterinin istediklerini karşıladık mı? Burada yalnız geçerleme yapılır. Hoca işlemsel kullanımdaki testi onaylama testi (acceptance test) olarak da andı.
- Alt basamakların hepsinde yalnız doğrulama yapılır.

V model şelale (waterfall) modeline göre tanımlanmıştır: Önce yazılımın geliştirilmesi bitirilir, sonra test adımları aşağıdan yukarı, geliştirme adımlarına uygun şekilde yapılır. Alçak düzey tasarımda yapılanlar bileşen testinde, genel (mimari) tasarımda yapılanlar bütünlük testinde, yazılım gereksinim tanımında yapılanlar sistem testinde, müşterinin istedikleri işlemsel kullanımda sınanır.

Hoca aynı ilişkiyi, yazılım mühendisliği kitabında daha önce kullandığı başka bir şekille de gösterdi: Birim testi, bütünlük testi, sistem testi ve onaylama, sistem geliştirme aşamalarıyla (sistem gereksinimleri, tasarım, kodlama) eşleştiriliyordu. Ama V model şeklinin daha iyi anlattığını, diğerinin daha zor anlaşıldığını söyledi ve V modeli göz önünde tutmayı önerdi.

> **Hocanın vurgusu:** V model şeklini gözünüzün önünde tutmakta fayda var; doğrulamanın ve geçerlemenin hangi adımda yapıldığını bu şekil daha iyi gösteriyor.

12. Test Süreci Adımları ve Birim Testi

Hoca test süreci adımlarını sırayla birim testi, bütünlük testi, sistem testi ve onaylama (işlemsel kullanım) testi olarak göreceklerini söyledi. Regresyon testi, kabul testi, beta testi gibi alt test tanımlarına bunlardan sonra değinildi.

Testleri kimin yaptığı da şekilde gösteriliyordu. Şekilde mavi gösterilen birim testleri, yazılım geliştirme sırasında geliştiricilerin yaptığı aktivitelerdir. Bütünlük testini ve sistem testini test grubu yapar. İşlemsel kullanım aşamasındaki testlerin bir kısmını test grubu, bir kısmını potansiyel kullanıcılar yapar.

12.1 Birim Testi Nedir?

Birim testi (unit test), yazılım tasarımının en küçük birimi olan **modül** üzerinde uygulanır. Ayrıntılı tasarım tanımlarına dayanarak modülün içi değerlendirilir. Bu tanımlar şunlardır: prosedür tasarımı, veri tabanı tasarımı ve ara birim (interface) tasarımı. Bunların hepsi kontrol edilerek modül yapıları test edilir.

Birim testinde genellikle **saydam kutu testi** (white box test) uygulanır. Hoca "white box" için bilerek "saydam" sözcüğünü kullandığını söyledi: Saydam kutu testi, yazılım biriminin iç yapısının incelenip değerlendirilmesidir; yazılımın içi görünür. Saydam kutu ve kara kutu test tiplerinin ayrıntıları süreç tanımlarından sonra işlenecektir.

12.2 Birim Testini Kim Yapar?

Birim testini yazılım geliştiriciler yapar. Ekipteki kişiler geliştirdikleri modüllerin testini, iş test ekibine gitmeden yapabilir. Ayrıntılı tasarımda oluşturulan algoritmaları ve veri tabanı tasarımlarını kullanırlar. Kontrol yollarını bildikleri için test verisini bütün kontrol yollarını deneyecek şekilde seçebilirler. Herkes kendi yazdığı birimi test ettiği için birim testleri çok sayıda modül üzerinde paralel (eş zamanlı) yürür.

12.3 Birim Testinde Sınanan Noktalar

- Modülün ara birimleri (interface).
- Yerel veri yapıları.
- Kontrol yapılarındaki ana yollar.
- Hata işleme yolları.
- Modül sınırları: modülün diğer modüllerden ayrıldığı yerler.

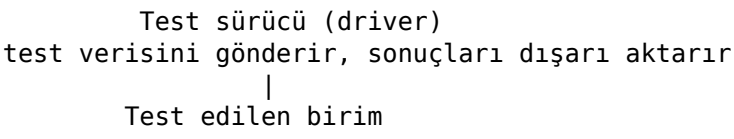
  Hocanın vurgusu: "Hata varsa kenarlarda, yani sınırlarda aramak lazım" diye bir kanı vardır. Bu yüzden birim testinde sınır değerlere mutlaka bakılır.

12.4 Test Sürücü ve Koçan

Birim testinin en basit hali şudur: Test edilen birime bir girdi verilir, bir sonuç çıkar ve sonucun doğru olup olmadığına bakılır. Birimi farklı verilerle çalıştırmak isteriz. Bu veriler test senaryoları (test case) şeklinde belirlenir. Test edilen birim bir modül, bir fonksiyon ya da bir prosedür olabilir; veriler ona doğrudan aktarılabilir.

Ama birim her zaman bağımsız değildir. Bu durumda iki yapay modül türü kullanılır:

- **Test sürücü (test driver):** Test verisini test edilecek modüle doğrudan gönderemeyebiliriz, çünkü bu modülü çağıran bir üst modül ya da prosedür vardır. Veri onun aracılığıyla geliyorsa o üst modülü temsil eden yapay bir test sürücü kullanılır. Test sürücü, test edilen birimden elde edilen sonuçları da dış ortama aktarır.
- **Koçan (stub):** Test edilen birimin çağırdığı alt modüller varsa (bir ya da birden fazla) bunları temsil eden yapay modüllerdir. Çağrılan alt modüllerin vereceği veriyi bu yapay modüller sağlar.



```
        /          |          \
   Koçan 1     Koçan 2     Koçan 3
  (stub: çağrılan alt modüllerin yerine geçen yapay modüller)
```

Hocanın vurguladığı kural şudur: Modül bağımsız değilse, yani modülü çağıran bir modül ya da modülün içinden çağrılan alt programlar varsa, her testte test sürücü ve/veya koçan yazılımı olmalıdır.

Çok fazla alt programı olmayan, dışarıdan veri alan küçük bir program için birim testinin hedefi basittir: Test senaryoları test edilen birime uygun girdiler olarak verilir, sonuçlar bulunur ve sonuçların elle hesaplananlarla aynı olup olmadığına bakılır.

12.5 Birim Test Araçları

Gerçek hayatta birim testinin daha sağlıklı ve daha hızlı yürümesi için araçlar (tool) kullanılır. Bu araçlar test senaryolarını oluşturmaya, modüle uygun test verisini seçmeye, testleri çalıştırmaya ve gerekirse çağrılan alt programların yerine geçen yapay alt programları oluşturmaya yardım eder. Hocanın örneği Java için JUnit'tir.

> **Ek bilgi (derste anlatılmadı):** JUnit, Java için büyük olasılıkla en yaygın birim test çerçevesidir. Her test senaryosu bir test metodu olarak yazılır ve beklenen sonuç ile gerçek sonuç bir doğrulama (assert) ifadesiyle karşılaştırılır. Bu, hocanın anlattığı "sonucu elle hesaplanan sonuçla karşılaştırma" işinin otomatikleştirilmiş halidir.

12.6 Test Senaryosu (Test Case)

Test senaryosu, belirli bir program yolunu işletmek ya da belirli bir gereksinime uygunluğu onaylamak için düzenlenen bir dizi test verisi ve bunlara ilişkin işlemlerden oluşur.

- **İç test (saydam kutu testi)** yapılıyorsa test verisi bir program akışı, yani kontrol akışı yolunu denemek için seçilir.
- **Dışarıdan, kullanıcının baktığı gibi** test yapılıyorsa test verisi kullanıcının belirlediği bir gereksinime uygunluğu sınamak için seçilir.

Test programları da diğer yazılımlar gibi geliştirilir. Hazır alınabilir ya da ekip tarafından yazılabilir. Bunlar test edilecek modüle test verisini gönderen ve sonuçları daha kolay almayı sağlayan programlardır. Test sürücü ve koçanların geliştirilmesinde de bu araçlardan faydalanılır.

13. Bütünlük Testi ve Regresyon Testi

13.1 Bütünlük Testi

Bütünlük testi (integration test), test edilmiş modüllerin birleştirilerek test edilmesidir. Bütün modüller test edildikten sonra yapılabilir ya da test edilen modüllerin bir kısmı birleştirilerek yapılabilir. Büyük resme yukarıdan bakarak, yukarıdan aşağı doğru birleştirme yapmak da mümkündür. Bu yüzden bütünlük testi farklı stratejilerle yapılır:

- Yukarıdan aşağıya (top-down).
- Aşağıdan yukarıya (bottom-up).
- Sandviç (sandwich): ikisinin birlikte kullanıldığı yaklaşım. Genellikle en çok bu kullanılır.

Bütünlük testinin ayrıntıları bir sonraki derste işlenecektir.

13.2 Regresyon (Tamamlama) Testi

Şekilde bütünlük testinin yanında regresyon testi de yer alır. Hoca bunu "tamamlama testi" diye Türkçeleştirdi. Bütünlük testi sırasında birleştirilmiş modüllerden birinde bir değişiklik ya da düzeltme yapılırsa, bu düzeltmeden sonra değişikliğin etkilediği bütün modüller ve yazılımın tamamı yeniden test edilmelidir. Buna regresyon testi denir.

Birim testinde olduğu gibi bütünlük testinde ve regresyon testinde de CASE araçları ve test araçları kullanılır.

14. Kabul, Alfa, Beta ve Teşhis Testi

Hoca bu kısımda slayttaki terimleri yine düzelttiğini söyledi: En üstte yalnız geçerleme, sistem testi düzeyinde hem geçerleme hem doğrulama, altta yalnız doğrulama yapılır.

Sistem testinden sonra kabul (onaylama) testi yapılır. Bu süreçte alfa testi ve beta testi yer alır. İşlemsel kullanım sırasında hata çıkarsa teşhis testi yapılır.

- **Kabul testi (acceptance test):** Teslim sırasında kullanıcının yaptığı testtir. Hoca bunu onay ya da onaylama testi olarak da andı.
- **Alfa testi:** Sistem testinden sonra, kullanıcı kabulünden önce yapılır. Yazılım evinde oluşturulan bir grup, kullanıcının yerine geçerek yazılımı kullanıcı bakış açısıyla test eder.
- **Beta testi:** Kullanıcı kabulünü geçtikten sonra yazılım potansiyel kullanıcılara dağıtılır. Onlardan hata bulmaları ve kullanıma geçmeleri istenir. Bu aşamada yazılım işlemsel kullanıma alınmış olur.
- **Teşhis testi:** İşlemsel kullanım sırasında bir hata meydana gelirse hatayı kontrol etmek için yeniden yapılan testtir.

15. Birim Testi Üzerine Soru-Cevap

Dersin sonunda hoca birim testiyle ilgili birkaç soru daha sordu. Ses sorunu olan öğrencilerin cevaplarını sohbet penceresine yazmasına izin verdi.

Soru: Koçan (stub) nedir? **Cevap:** Testlerde çağrılan alt programlar varsa onların yerini tutan yapay modüllerdir. Henüz test edilmemiş modüllerin yerine konabilir; veri tabanı bağlantılarının yerine de konabilir. Birim testlerinde kullanılan yapay test öğeleridir.

Soru: Test senaryosu (test case) nedir? **Cevap:** Bir kodu sınarken ya da gereksinimlerden birini test ederken uygun verilerle test yapılması ve bu veri gruplarının oluşturulmasıdır. Test senaryosu kontrol akışından çıkan bir yol olabilir, bir veri grubu olabilir ya da bir program için kullanıcının istediği özelliklerin sınanması olabilir.

Örnek Sorular

Bu derste ekrandan soru gösterilmedi. Aşağıdaki sorular hocanın derste sözlü sorduğu sorulardır.

1. (derste sözlü, 01:14:59) İnceleme (inspection) ile gözden geçirme (review) arasındaki farkları söyleyiniz. Önceden hazırlık yapılır mı, rapor hazırlanır mı gibi ölçütlere göre karşılaştırınız.

> **Çözüm (derste verilmedi):** Derste öğrenciler yalnız “inceleme daha ayrıntılıdır” cevabını verdi, hoca ayrıca düzeltme yapmadı. Seçenek matrisine (Bölüm 2) göre: İncelemede bir lider, toplantı takvimi ve kayıtçı vardır. Gözden geçirenler mutlaka ön hazırlık yapar. Hataları bulmak için kontrol listesi kullanılır, bulunan hatalar sınıflanır ve sayılır. Yazılımcı sunum yapmaz, doğrudan ürün incelenir. Formal olmayan gözden geçirmede lider, kontrol listesi ve hata sınıflaması yoktur; hazırlık, sunum ve kayıtçı belki olur. İnceleme kodu çalıştırmadan yapılır (statik doğrulama) ve sonunda teknik inceleme özet raporu hazırlanır.

2. (derste sözlü, 01:18:02 ve 01:19:01) Statik doğrulama (static verification) ne demektir? Gözden geçirme, inceleme ve testten hangisi statik doğrulamadır?

> **Çözüm (derste verildi):** Programı çalıştırmadan yapılan doğrulamadır. İnceleme statik doğrulamadır.

3. (derste sözlü, 01:19:42) Dinamik doğrulama (dynamic verification) hangisidir?

> **Çözüm (derste verildi):** Testtir. Birim testi doğru ama eksik bir cevaptır: Bütünlük ve sistem testi de program parçasını ya da bütününü çalıştırdığı için dinamik doğrulamadır.

4. (derste sözlü, 02:25:33) Koçan (stub) nedir?

Çözüm (derste verildi): Test edilen birimin çağırdığı alt programların ya da henüz test edilmemiş modüllerin yerine geçen yapay modüllerdir. Veri tabanı bağlantılarının yerine de konabilir.

5. (derste sözlü, 02:27:53) Test senaryosu (test case) nedir?

Çözüm (derste verildi): Belirli bir program yolunu işletmek ya da belirli bir gereksinime uygunluğu onaylamak için hazırlanan test verileri ve ilgili işlemlerdir. Saydam kutu testinde veri bir kontrol akışı yolunu denemek için, kullanıcı gözüyle testte bir gereksinimi sınamak için seçilir.

Dersin Toparlaması

Bu derste kalite sağlama aktivitelerinin statik tarafı tamamlandı ve dinamik tarafa, yani teste geçildi. Gözden geçirme tekrar edildi: amacı, toplantı rolleri, formal olmayan gözden geçirme, walkthrough ve inspection arasındaki farklar ve süreç basamaklarının her birinde neye bakıldığı. Ardından inceleme ayrıntılandı: İnceleme kodu çalıştırmadan, bütün yarı ürünler üzerinde yapılan statik doğrulamadır. Genel inceleme süreci üç aşamadan oluşur. Fagan yöntemi bu süreci planlama, planın gözden geçirilmesi, kişisel kod inceleme, inceleme toplantısı, düzeltme ve tamamlama-raporlama adımlarına böler. Süreç teknik inceleme özet raporu ve üç olası kararla biter.

Dersin ikinci yarısında doğrulama “ürünü doğru yolla üretiyor muyuz?”, geçerleme “doğru ürünü mü ürettik?” olarak tanımlandı. V model, sol koldaki geliştirme adımlarını sağ koldaki test adımlarıyla eşleştirdi: alt basamaklarda yalnız doğrulama, sistem testinde ikisi birden, işlemsel kullanımda yalnız geçerleme yapılır. Birim testi, test sürücü ve koçan, test senaryosu ve araçlarıyla anlatıldı; bütünlük testi stratejileri, regresyon, kabul, alfa, beta ve teşhis testleri kısaca tanımlandı. Bir sonraki derste bütünlük testinin ayrıntılarından devam edilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.